



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

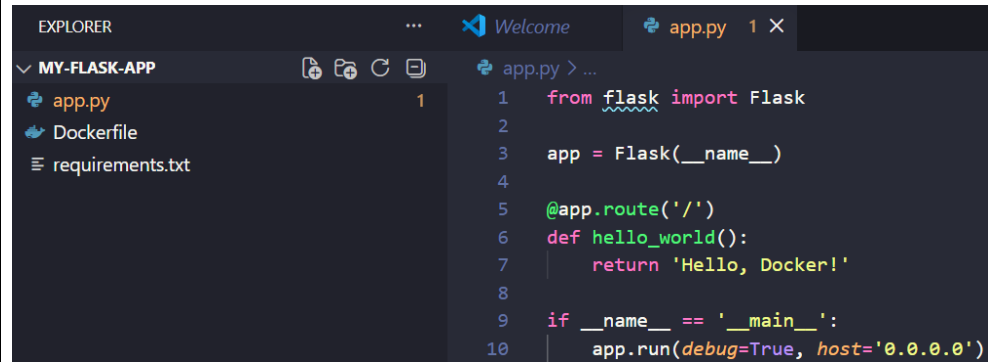
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 08

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Ria Vinod
PRN No.	23070122179
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	23 rd July, 2026
Title of Assignment:	Dockerfile & Image Build
Theory:	Docker is a containerization platform that enables applications and their dependencies to be packaged into lightweight, portable containers. A Dockerfile contains the instructions required to build a Docker image, while a Docker image serves as a template for creating containers. A Docker container is a running instance of an image that provides an isolated environment for executing applications consistently across different systems. In this experiment, a simple Flask web application was containerized by creating the required application files, building a Docker image, running the container, and verifying the application through a web browser. Commands Used • mkdir my-flask-app – Created a new directory for the Flask application. • cd my-flask-app – Navigated to the project directory. • docker build -t my-flask-app . – Built a Docker image named my-flask-app from the Dockerfile. • docker build --no-cache -t my-flask-app . – Rebuilt the Docker image without using cached layers after updating the dependencies. • docker run -p 5000:5000 my-flask-app – Started the Docker container and mapped port 5000 of the container to port 5000 on the host machine for browser access.

Implementation:

Step 1: Create the Flask Application

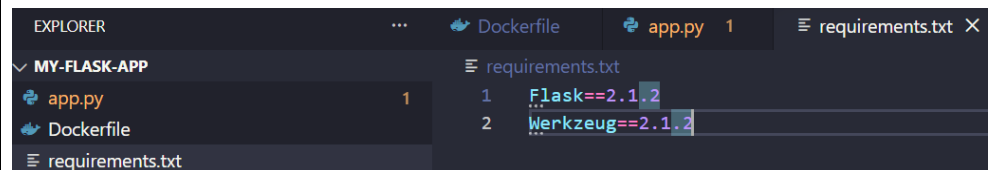


The screenshot shows the VS Code interface. The Explorer sidebar on the left shows a project named 'MY-FLASK-APP' containing three files: 'app.py', 'Dockerfile', and 'requirements.txt'. The Editor window on the right is open to 'app.py' and contains the following Python code:

```
1 from flask import Flask
2
3 app = Flask(__name__)
4
5 @app.route('/')
6 def hello_world():
7     return 'Hello, Docker!'
8
9 if __name__ == '__main__':
10     app.run(debug=True, host='0.0.0.0')
```

Created Flask Application — Created the my-flask-app project directory and added the required application files. Implemented the app.py file containing a simple Flask web application that displays the message "Hello, Docker!" when accessed.

Step 2: Create the Requirements File

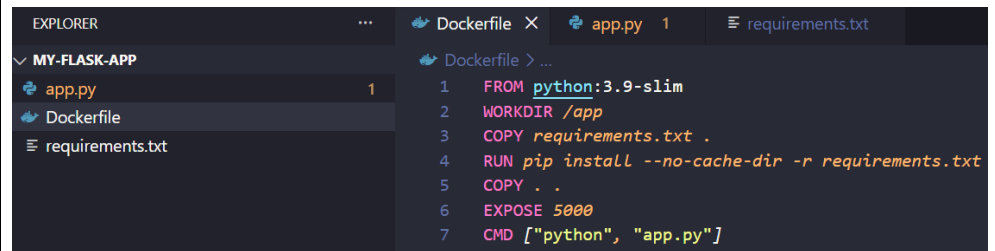


The screenshot shows the VS Code interface. The Explorer sidebar on the left shows the 'MY-FLASK-APP' project with 'app.py', 'Dockerfile', and 'requirements.txt'. The Editor window is open to 'requirements.txt' and contains the following text:

```
1 Flask==2.1.2
2 Werkzeug==2.1.2
```

Created Requirements File — Created the requirements.txt file specifying the Flask dependency required to install the necessary packages for the Flask application.

Step 3 – Create the Dockerfile

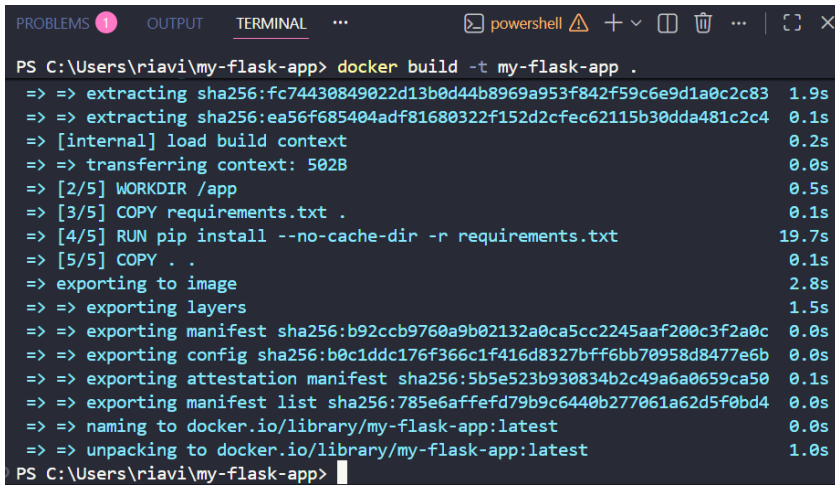


The screenshot shows the VS Code interface. The Explorer sidebar on the left shows the 'MY-FLASK-APP' project with 'app.py', 'Dockerfile', and 'requirements.txt'. The Editor window is open to 'Dockerfile' and contains the following Dockerfile instructions:

```
1 FROM python:3.9-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY . .
6 EXPOSE 5000
7 CMD ["python", "app.py"]
```

Created Dockerfile — Created the Dockerfile specifying the Python base image, working directory, application dependencies, exposed port, and startup command required to containerize the Flask application.

Step 4 – Build the Docker Image



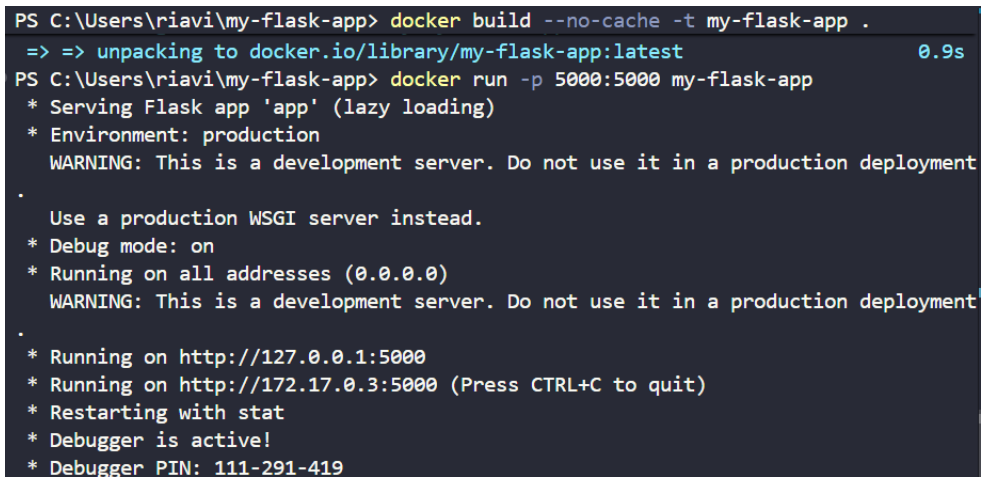
```

PS C:\Users\riavi\my-flask-app> docker build -t my-flask-app .
=> => extracting sha256:fc74430849022d13b0d44b8969a953f842f59c6e9d1a0c2c83 1.9s
=> => extracting sha256:ea56f685404adf81680322f152d2cfec62115b30dda481c2c4 0.1s
=> [internal] load build context
=> => transferring context: 502B 0.2s
=> [2/5] WORKDIR /app 0.5s
=> [3/5] COPY requirements.txt . 0.1s
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt 19.7s
=> [5/5] COPY . . 0.1s
=> exporting to image 2.8s
=> => exporting layers 1.5s
=> => exporting manifest sha256:b92ccb9760a9b02132a0ca5cc2245aaf200c3f2a0c 0.0s
=> => exporting config sha256:b0c1ddc176f366c1f416d8327bff6bb70958d8477e6b 0.0s
=> => exporting attestation manifest sha256:5b5e523b930834b2c49a6a0659ca50 0.1s
=> => exporting manifest list sha256:785e6affefd79b9c6440b277061a62d5f0bd4 0.0s
=> => naming to docker.io/library/my-flask-app:latest 0.0s
=> => unpacking to docker.io/library/my-flask-app:latest 1.0s
PS C:\Users\riavi\my-flask-app>

```

Built Docker Image — Successfully built the Docker image named my-flask-app using the Dockerfile. Docker completed all build stages and created the container image successfully.

Step 5 – Run the Docker Container



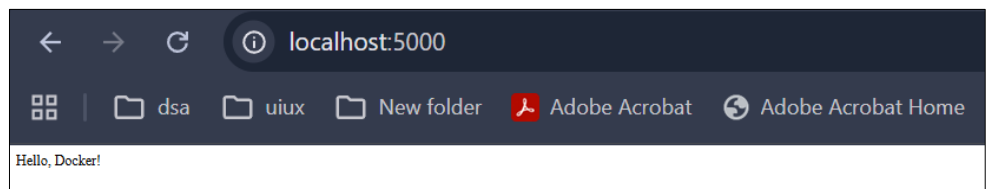
```

PS C:\Users\riavi\my-flask-app> docker build --no-cache -t my-flask-app .
=> => unpacking to docker.io/library/my-flask-app:latest 0.9s
PS C:\Users\riavi\my-flask-app> docker run -p 5000:5000 my-flask-app
* Serving Flask app 'app' (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment
.
  Use a production WSGI server instead.
* Debug mode: on
* Running on all addresses (0.0.0.0)
  WARNING: This is a development server. Do not use it in a production deployment
.
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.3:5000 (Press CTRL+C to quit)
* Restarting with stat
* Debugger is active!
* Debugger PIN: 111-291-419

```

Executed Docker Container — Successfully started the Docker container and launched the Flask application. The application is running on port 5000 and is accessible through the host machine.

Step 6: Verify the Flask Application



The screenshot shows a web browser window with the address bar set to `localhost:5000`. The browser's address bar and tabs are visible at the top. The main content area of the browser displays the text "Hello, Docker!" in a simple font.

Verified Dockerized Flask Application — Opened the Flask application in a web browser using <http://localhost:5000> and successfully verified that the Docker container served the "Hello, Docker!" webpage.